EAST WEST UNIVERSITY

Project Report

Implementation of Cache Management Policies in a Fixed-Size Cache to Analyse Cache Pollution and Thrashing in C.

Course Title: Computer Architecture

Code: CSE360

Semester: Spring 2026

Section: 1

Submitted To:

Dr. Md. Nawab Yousuf Ali

Professor

Department of Computer Science & Engineering

Submitted by

Name	ID
Shairin Akter Hashi	2022-2-60-102
Zihad Khan	2022-2-60-107

Date of Submission: 21.04.2026

Table of Contents

1. Title

Implementation of Cache Management Policies in a Fixed-Size Cache to Analyse Cache Pollution and Thrashing in C.

This project focuses on simulating cache behaviours using two widely used replacement policies, namely Least Recently Used (LRU) and First In First Out (FIFO). The system is designed to operate with a fixed cache size and aims to analyse how different policies impact performance, cache efficiency, and memory usage.

2. Objective

The aim of the project is to construct and create a cache management system with C code that simulates different cache replacement methods and assesses how useful they are.

The project's specific goals are as follows:

- Implement a fixed amount of cache to hold **N number of pages.**
- Develop and simulate two different cache replacement plans.
    1. Least Recently Used (**LRU**)
    2. First In First Out (**FIFO**)
- Perform a performance measure of the cache by calculating and comparing how effective the cache is by determining its hit ratio versus their miss ratio.
- Provide a **step-by-step** simulation demonstrating the cache when **utilizing** caches at each step to help gain better understanding of how caching works.
- Examine how **cache access patterns** will impact the performance of the cache.
- Study the **impact** of cache replacement methodologies on cache pollution and thrashing.

This project performs LRU & FIFO cache replacement in a fixed-size cache, in that LRU cache replacement minimizes cache pollution & thrashing by maintaining only recently used pages. Furthermore, using comparative methodology illustrates **how caching methodologies impact performance.**

3. Theory

This section explains the basic concepts related to cache memory and replacement policies.

3.1 Cach Memory

Cache devices are small, fast pieces of **hardware** that store data that is frequently accessed. When your computer looks for the same data in the future, the cache will provide it quickly. The cache sits **between** the **CPU** and the **main memory**. The main purpose of cache memory is to:

- Reduce memory access time.
- Improve overall system performance.
- Minimize delays in data retrieval.

Since a cache **has limited capacity**, it cannot store all the data that could potentially be found via a computer. Because of this **limitation,** it is important to **manage** the cache properly.

3.2 Cache Replacement Policies

When the cache is full and a new page needs to be added, there is a decision that must be made about which existing page will be removed. The decision on which page to remove is based on the cache replacement policy in place.

3.2.1. FIFO (First In First Out)

FIFO is one of the simpler cache replacement policies. It will remove the very first page that was loaded into the cache and add the new page into the empty slot. FIFO has a queue-like structure and does not consider how frequently, or recently, the page has been used.

Advantages:

- It is **easy** to implement.
- It has a **low computational** overhead.

Disadvantages:

- It may **remove** important pages that have been previously accessed.
- May lead to **sub-optimal** performance in certain situations.

3.2.2. LRU (Least Recently Used)

LRU is a more intelligent cache replacement policy. It will remove the page that has not been accessed in the longest amount of time and, therefore, assume that recently accessed pages are more likely to be accessed again.

Advantages:

- Provides superior performance under many **real-world conditions**.
- **Minimizes** unnecessary page replacements.

Disadvantages:

- Slightly more **complex** to implement than FIFO.
- Requires **maintenance of a record** of previously accessed pages.

3.3 Cache Pollution

Cache pollution happens when data that rarely gets used takes up space in the cache. This makes caching less efficient because some of the useful data will get deleted to make room for the useless or less useful data.

Cache pollution means the cache is filled with "useless" or less useful pages.

Using LRU will help reduce cache pollution by keeping pages that have been used recently in the cache where they are most likely to be needed again.

3.4 Thrashing

In computing, thrashing describes a scenario where a computer performs too many cache misses and continually replaces pages.

Because of this:

- Pages are being replaced **too frequently** and as such.
- System performance is **degraded**.

So, in a typical case of thrashing, the user is **experiencing symptoms** of low performance due to.

- Too **small** of a cache size.
- **Poor** access patterns for pages.

The use of LRU **(Least Recently Used)** is one method for reducing thrashing by allowing the computer to use previous usage history to make more intelligent decisions about which pages to load into memory.

3.5 Working Principle of This Project

Fixed-size arrays are used here to simulate a cache. An input string of pages is provided, each of which is processed in a sequential fashion.

A page can either be deemed a "**Hit**" if it is in cache. Alternatively, if it is not contained in cache, it would count as a "**Miss**".

When a page miss occurs with respect to a full cache, the cache selectively replaces the oldest page according to the FIFO policy or it replaces the least recently used page according to the LRU policy.

In addition, each access is tracked for:

- a total number of hits made by the user.
- a total number of misses made by the user.
- the percentage of hits made versus misses made (hit ratio).

We can find the best performing algorithm in each situation by comparing these numbers.

This section explains the basic concepts related to cache memory and replacement policies.

3.1 Cach Memory

Cache devices are small, fast pieces of hardware that store data that is frequently accessed. When your computer looks for the same data in the future, the cache will provide it quickly. The cache sits between the CPU and the main memory. The main purpose of cache memory is to:

- Reduce memory access time .
- Improve overall system performance.
- Minimize delays in data retrieval.

Since a cache has limited capacity, it cannot store all the data that could potentially be found via a computer. Because of this limitation, it is important to manage the cache properly.

3.2 Cache Replacement Policies

When the cache is full and a new page needs to be added, there is a decision that must be made about which existing page will be removed. The decision on which page to remove is based on the cache replacement policy in place.

3.2.1. FIFO (First In First Out)

FIFO is one of the simpler cache replacement policies. It will remove the very first page that was loaded into the cache and add the new page into the empty slot. FIFO has a queue-like structure and does not consider how frequently, or recently, the page has been used.

Advantages:

- It is easy to implement.
- It has a low computational overhead.

Disadvantages:

- It may remove important pages that have been previously accessed.
- May lead to sub-optimal performance in certain situations.

3.2.2. LRU (Least Recently Used)

LRU is a more intelligent cache replacement policy. It will remove the page that has not been accessed in the longest amount of time and, therefore, assume that recently accessed pages are more likely to be accessed again.

Advantages:

- Provides superior performance under many real-world conditions.
- Minimizes unnecessary page replacements.

Disadvantages:

- Slightly more complex to implement than FIFO.
- Requires maintenance of a record of previously accessed pages .

3.3 Cache Pollution

Cache pollution happens when data that rarely gets used takes up space in the cache. This makes caching less efficient because some of the useful data will get deleted to make room for the useless or less useful data.

Cache pollution means the cache is filled with "useless" or less useful pages.

Using **LRU** will help **reduce cache pollution** by keeping pages that have been used recently in the cache where they are most likely to be needed again.

3.4 Thrashing

In computing, thrashing describes a scenario where a computer performs too many cache misses and continually replaces pages.

Because of this:

- Pages are being replaced too frequently and as such.
- System performance is degraded.

So, in a typical case of thrashing, the user is experiencing symptoms of low performance due to.

- Too small of a cache size.
- Poor access patterns for pages.

The use of LRU (Least Recently Used) is one method for reducing thrashing by allowing the computer to use previous usage history to make more intelligent decisions about which pages to load into memory.

3.5 Working Principle of This Project

Fixed-size arrays are used here to simulate a cache. An input string of pages is provided, each of which is processed in a sequential fashion.

A page can either be deemed a "**Hit**" if it is in cache. Alternatively, if it is not contained in cache, it would count as a "**Miss**".

When a page miss occurs with respect to a full cache, the cache selectively replaces the oldest page according to the FIFO policy or it replaces the least recently used page according to the LRU policy.

In addition, each access is tracked for:

- a total number of hits made by the user.
- a total number of misses made by the user.
- the percentage of hits made versus misses made (hit ratio).

We can find the best performing algorithm in each situation by comparing these numbers.

4. Design

This section briefly describes the overall design, main parts, and working steps of the Cache Memory Management System.

4.1 Overall System Design

This system is a menu-driven console program, which is designed to simulate the cache memory management process. The user first selects the manual input or random generation option. Then the page sequence and cache size are input. The program then applies the **LRU** and **FIFO** algorithms on the same input.

The system is designed in such a way that the current state of the cache and the **hit** or **miss** status are displayed at each step. As a result, the user can easily understand how the page replacement is happening. At the end, the program displays the **total hits**, **misses** and **hit ratio**, so that the performance of the **two algorithms** can be compared.

4.2 Functional Modules

This system consists of several main modules, where each module performs a specific task.

1. **Input Module:** Receives page sequence and cache size from the user. There are two options -
    - Manual input.
    - Random generation.
2. **Manual Input Module:** User can enter the page number and page sequence manually to test the system.
3. **Random Generation Module:** User can generate random page sequence if needed, which makes testing easier.

4. **LRU Simulation Module:** Updates cache using LRU algorithm and shows page replacement at each step.
5. **FIFO Simulation Module:** Updates cache according to FIFO algorithm and replaces oldest page.
6. **Display Module:** Shows current cache status and hit or miss status at each step for both LRU and FIFO.
7. **Result Module:** Finally calculates total hits, misses and hit ratio and compares the performance of the two algorithms.
8. **Repeat Module:** Allows the user to rerun the program if desired.

In this way, the module-based design makes the system easy to understand and maintain.

4.3 Data Structures Used

Page Array: Used to store **page reference string**.

Cache Array: Used to store **current pages** in cache memory.

Time Array: Used to **track** when a page was **last used** in LRU algorithm.

Pointer Variable: Used to determine **next replacement position** in FIFO algorithm.

Counter Variables: Used to keep **hit**, **miss** and **other necessary** counts.

4.4 Working Procedure

This system works by following certain steps. The steps are summarized below:

- First, when the program is launched, a **menu** is displayed in front of the user.
- The user selects **manual input** or **random generation** option.
- Then the **number of pages** and **page sequence** are **input** (or **generated**).
- The user determines the **cache size**.
- The program first shows a **step-by-step cache simulation** using the **LRU algorithm**.
- Then the simulation is **run** in the same way using the **FIFO algorithm**.
- At each step, the **cache status** and **hit or miss** status are **displayed**.
- Finally, the **total hits**, **misses** and **hit ratio** are calculated and the results of **the two algorithms** are shown.

- If the user wants, the program can be **re-run**, or if not, the program is **stopped**.

By following these steps, the system completes the work easily and consistently.

4.5 Algorithm Design

In this system, LRU (Least Recently Used) and FIFO (First In First Out) algorithms have been designed to manage **cache page replacement**. Both algorithms process the page **reference string sequentially** and check whether the page is **present** in the cache for each **page request**. If the page already exists in the cache, then it is counted as a hit. On the other hand, if the page is not in the cache, then it is considered a **miss**, and a **page replacement** is performed according to the policy of the respective algorithm.

4.5.1 LRU Algorithm Design

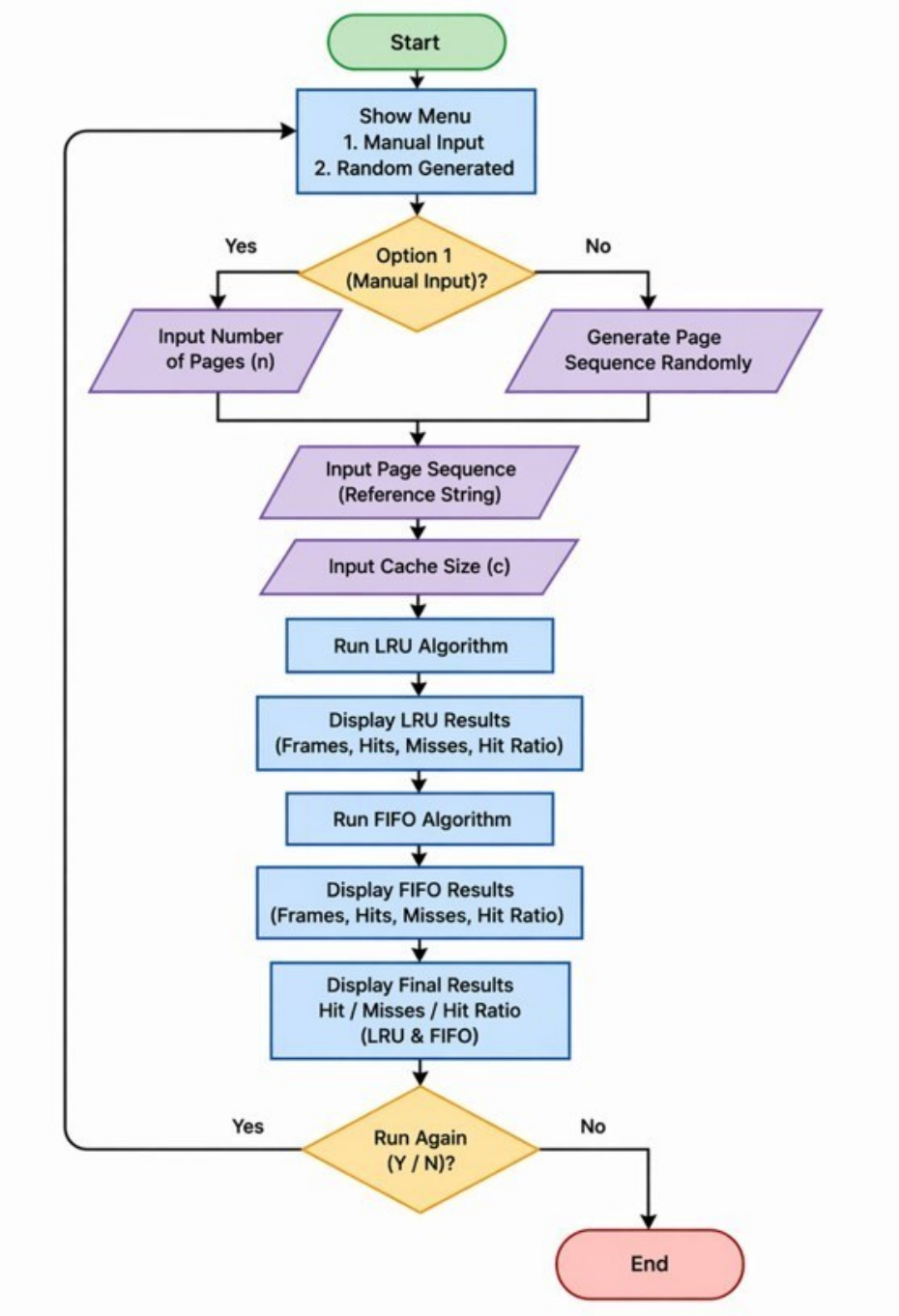
The LRU algorithm is designed in such a way that when the cache is full, the new page is replaced with the page that has not been used for the **longest time**. For this purpose, the last usage information of each cached page is **stored**. As a result, the system can easily determine the **least** recently used page and perform the **replacement**.

4.5.2 FIFO Algorithm Design

The FIFO algorithm is designed in such a way that when the cache is full, the page that was **first entered** in the cache is **replaced first**. Here, the recent usage of the page is not considered; Rather, the insertion order is followed. In this method, the **oldest page** is removed, and the **new page** is added to the cache.

The design of these **two algorithms** is structured in such a way that their operation and performance on the same page sequence can be clearly observed and compared.

4.6 Program Flow



5. Implementation

5.1 Implemented Functions

1. **Page Sequence Handling:**
   - generateRandom(int pages[], int n) generates random page references in the range **0 to 9**.
   - Manual input mode accepts **user-defined page** reference strings.
2. **LRU Function:**
   - LRU(int pages[], int n, int size) performs **Least Recently** Used replacement.
   - It uses local arrays cache[] and **time[]** and a logical time counter to **track recent usage**.
3. **FIFO Function:**
   - FIFO(int pages[], int n, int size) performs **First-In First-Out** replacement.
   - It uses **local cache[]** and **circular pointer** front for deterministic replacement order.
4. **Step-by-Step Trace in main():**
   - Prints table headers and per-step state for LRU and FIFO.
   - Displays current page, cache content, and hit or miss status at each step.
5. **User Interface in main():**
   - main() provides an **interactive loop**.
   - Menu selection (**Manual Input** or **Random Generate**).
   - Number of pages and **cache size** input.
   - **Repeat** execution prompt (Do you want to **run** again?).

5.2 Hierarchical Relationships

- 'main()' coordinates the full **workflow** and **calls** all core modules.
- 'main()' calls '**generateRandom()**' only when **random mode** is selected.
- 'main()' executes inline step-by-step LRU and FIFO **tracing** blocks.
- 'main()' then calls 'LRU()' and 'FIFO()' to produce final summarized metrics.
- 'LRU()' and 'FIFO()' are independent algorithm modules sharing the same input data.

5.3 Code Documentation

- Function names are descriptive and reflect algorithm intent.

- The code is segmented with clear section comments.
- random generation.
- LRU algorithm.
- FIFO algorithm.
- LRU or FIFO step-by-step simulations.
- Final results.
- Output formatting is table-like, making debugging and report verification easier.

5.4 System Requirements

- Compiler: **GCC** or **g++** (C/C++ standard toolchain).
- Operating System: Platform-independent; tested on Windows.
- Runtime: Console-based standard input or output.
- Memory Model: **Fixed-size** arrays with 'MAX = 100'.
- Dependencies: Standard libraries only ('stdio.h', 'stdlib.h', 'time.h').

5.5 Implemented Code

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX 100

// Generate random page sequence

void generateRandom(int pages[], int n) {
for(int i = 0; i < n; i++) {
pages[i] = rand() % 10;
    }
}
 // LRU
Algorithm
```

```
float LRU(int pages[], int n, int size) {
int cache[MAX], time[MAX];     int hit =
0, miss = 0, counter = 0;

    for(int i = 0; i < size; i++) {
cache[i] = -1;          time[i] = 0;
    }
    for(int i = 0; i < n; i++) {
int found = 0;          for(int j = 0; j
< size; j++) {              if(cache[j]
== pages[i]) {                  hit++;
counter++;                  time[j] =
counter;                  found = 1;
break;
            }
        }          if(!found) {
miss++;              int lru_index = 0;
for(int j = 1; j < size; j++) {
if(time[j] < time[lru_index])
lru_index = j;
            }
            cache[lru_index] = pages[i];
counter++;              time[lru_index] =
counter;
        }
    }
    printf("\n--- LRU Results ---\n");     printf("Hits = %d\nMisses =
%d\nHit Ratio = %.2f\n", hit, miss,
(float)hit/n);
return (float)hit/n;
}

// FIFO Algorithm
float FIFO(int pages[], int n, int size) {
int cache[MAX];     int front = 0;     int
hit = 0, miss = 0;
```

```
    for(int i = 0; i < size; i++)
cache[i] = -1;

    for(int i = 0; i < n; i++) {
int found = 0;
        for(int j = 0; j < size; j++) {
if(cache[j] == pages[i]) {
hit++;                  found = 1;
break;
            }
        }
        if(!found) {
cache[front] = pages[i];
front = (front + 1) % size;
miss++;
        }
    }
    printf("\n--- FIFO Results ---\n");
    printf("Hits = %d\nMisses = %d\nHit Ratio = %.2f\n", hit, miss,
(float)hit/n);
return (float)hit/n;
}  int
main() {
int
pages[MAX];
int n, size,
choice;
char again;

srand(time(0));
     do
{
        printf("\n===== Cache Memory Management System =====\n");
printf("1. Manual Input\n2. Random Generate\nChoose: ");         scanf("%d",
&choice);
```

```
        printf("Enter number of pages: ");
scanf("%d", &n);

        if(choice == 1) {
            printf("Enter page sequence:\n");
for(int i = 0; i < n; i++) {
scanf("%d", &pages[i]);
            }
}
else {

            generateRandom(pages, n);
printf("Generated Sequence:\n");
for(int i = 0; i < n; i++)
printf("%d ", pages[i]);
printf("\n");
        }
        printf("Enter cache size: ");
scanf("%d", &size);

        // ================== LRU STEP-BY-STEP ==================

        printf("\nStep-by-step LRU Simulation:\n");

        int cacheArr[MAX], timeArr[MAX];
int hit = 0, miss = 0, counter = 0;
for(int i = 0; i < size; i++) {
cacheArr[i] = -1;             timeArr[i] =
0;
        }
        printf("Step\tPage\tCache\t\tHit/Miss\n");

        for(int i = 0; i < n; i++) {
int found = 0;
```

```
            for(int j = 0; j < size; j++) {
if(cacheArr[j] == pages[i]) {
found = 1;                      hit++;
counter++;                      timeArr[j] =
counter;                    break;
                }
            }
            if(!found) {                miss++;
int lru_index = 0;                for(int j = 1; j <
size; j++) {                    if(timeArr[j] <
timeArr[lru_index])                         lru_index =
j;
                }

                cacheArr[lru_index] = pages[i];
counter++;                  timeArr[lru_index] =
counter;
            }
            printf("%d\t%d\t", i+1, pages[i]);
for(int k = 0; k < size; k++) {
if(cacheArr[k] != -1)
printf("%d ", cacheArr[k]);                 else
printf("- ");
            }

            if(found) printf("\tHit\n");
else printf("\tMiss\n");
        }

        // ================ FIFO STEP-BY-STEP ================

        printf("\nStep-by-step FIFO Simulation:\n");

        int fifoCache[MAX];
int frontPtr = 0;         int fifoHit
= 0, fifoMiss = 0;
```

```
        for(int i = 0; i < size; i++)
fifoCache[i] = -1;

        printf("Step\tPage\tCache\t\tHit/Miss\n");

        for(int i = 0; i < n; i++) {
int found = 0;

            for(int j = 0; j < size; j++) {
if(fifoCache[j] == pages[i]) {
found = 1;                      fifoHit++;
break;
                }
            }

            if(!found) {
                fifoCache[frontPtr] = pages[i];
frontPtr = (frontPtr + 1) % size;
fifoMiss++;
            }

            printf("%d\t%d\t", i+1, pages[i]);
for(int k = 0; k < size; k++) {
if(fifoCache[k] != -1)
printf("%d ", fifoCache[k]);
else                        printf("- ");
            }

            if(found) printf("\tHit\n");
else printf("\tMiss\n");
        }

        // ================= FINAL RESULTS =================
```

```
        float lru_ratio = LRU(pages, n, size);
float fifo_ratio = FIFO(pages, n, size);

        // Repeat option
        printf("\nDo you want to run again? (y/n): ");
scanf(" %c", &again);

    }
while(again == 'y' || again == 'Y');

    printf("\nProgram exited.\n");
return 0;
}
```

5.6 Code Explanation

1. **Global Settings and Data Structures**
   - 'MAX' defines the maximum supported number of pages and cache slots in arrays.
   - 'pages[]' stores the page reference string.
   - Cache arrays are initialized with '**-1**' to indicate **empty slots**.

2. **Random Sequence Generation**
   - 'generateRandom()' fills 'pages[]' with values between 0 and 9 using '**rand() % 10**'.
   - '**srand(time(0))**' in 'main()' ensures different random sequences in different runs.

3. **LRU Algorithm Logic**
   - For each page reference:
     - **Search** cache for a hit.
     - On hit, update page **timestamp** using '**counter**'.
     - On a miss, replace the cache entry with the smallest timestamp (least recently used).
   - Prints final hit or miss counts and ratio.

4. **FIFO Algorithm Logic**

- Maintains a **circular replacement** index ('front').
- For each page reference:
    - If hit, no replacement.
    - If you miss, replace the page at '**front**' and advance 'front'.
- Prints final **hit or miss counts** and ratio.

5. **Step-by-Step Simulation**

- 'main()' separately performs detailed traces for LRU and FIFO before final summary.
- At every step, it prints: ○ step number ○ current page ○ cache content ○ Hit or Miss
- This output supports debugging and correctness verification.

6. **User Interface and Control Flow**

- Menu-driven interactive loop with two input modes:
    - Manual sequence ○ Random sequence
- After one complete simulation, asks user whether to run again.

6. Debugging and Test-Run

This section explains the testing methodology used to validate correctness, consistency, and practical behaviour of the cache simulation program.

6.1 Testing Methodology

1. Tested both input modes:

- Manual input mode with **known sequences.**
- Random mode behaviour (validated by reusing generated sequence in manual mode).

2. Verified for each run:

- Correct step-by-step cache transitions.
- Correct hit or miss status at each step.
- Correct final totals and hit ratios for both LRU and FIFO.
- Repeated runs to validate loop behaviour and program stability.

6.2 Debugging

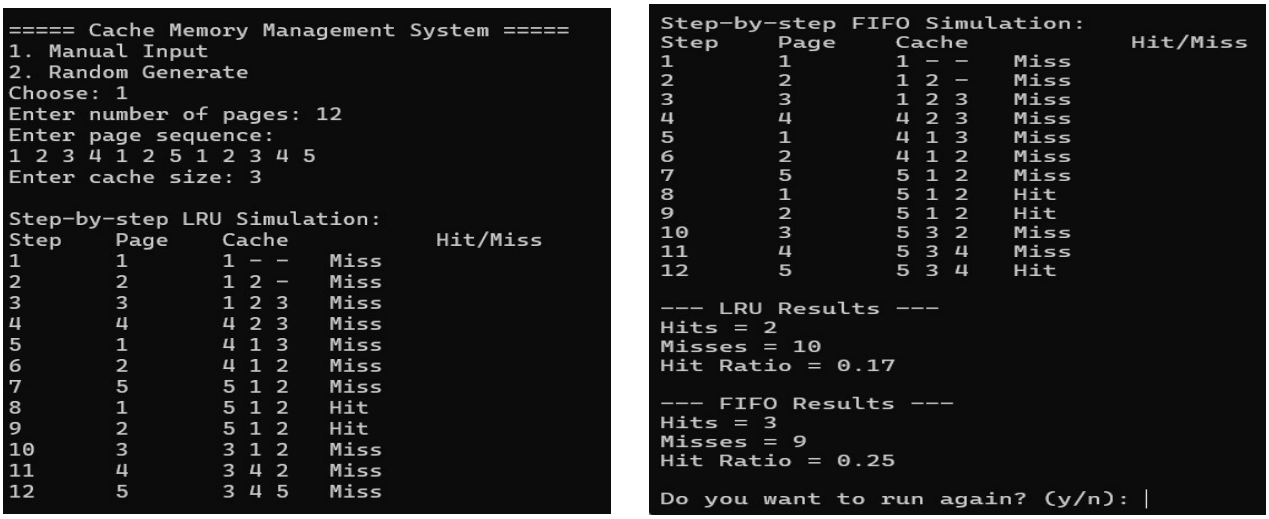
1. Build issue encountered.
2. ‘ld.exe: cannot open output file main.exe: Permission denied’
3. Cause:
   - ‘main.exe’ was locked or in use during recompilation.

Resolution:

- Execute existing binary directly or stop the running process before rebuilding.

6.3 Verified Test Runs

Test Case 1:



```
===== Cache Memory Management System =====
1. Manual Input
2. Random Generate
Choose: 1
Enter number of pages: 12
Enter page sequence:
1 2 3 4 1 2 5 1 2 3 4 5
Enter cache size: 3

Step-by-step LRU Simulation:
Step    Page    Cache           Hit/Miss
1       1       1 - -   Miss
2       2       1 2 -   Miss
3       3       1 2 3   Miss
4       4       4 2 3   Miss
5       1       4 1 3   Miss
6       2       4 1 2   Miss
7       5       5 1 2   Miss
8       1       5 1 2   Hit
9       2       5 1 2   Hit
10      3       3 1 2   Miss
11      4       3 4 2   Miss
12      5       3 4 5   Miss

Step-by-step FIFO Simulation:
Step    Page    Cache           Hit/Miss
1       1       1 - -   Miss
2       2       1 2 -   Miss
3       3       1 2 3   Miss
4       4       4 2 3   Miss
5       1       4 1 3   Miss
6       2       4 1 2   Miss
7       5       5 1 2   Miss
8       1       5 1 2   Hit
9       2       5 1 2   Hit
10      3       5 3 2   Miss
11      4       5 3 4   Miss
12      5       5 3 4   Hit

--- LRU Results ---
Hits = 2
Misses = 10
Hit Ratio = 0.17

--- FIFO Results ---
Hits = 3
Misses = 9
Hit Ratio = 0.25

Do you want to run again? (y/n):
```

Fig 1. Output of Test Case 1 for LRU and FIFO

Output Explanation:

Input:

Mode: Manual

Number of pages: 12

Page sequence: ‘1 2 3 4 1 2 5 1 2 3 4 5 ’

Cache size: 3

Process:

1. The program runs the same input through both replacement algorithms.
2. For each page reference, it checks whether the page already exists in cache.
    - If yes: count as ‘Hit’.
    - If not: count as ‘Miss’ and replace one page based on algorithm rule.
3. LRU rule:
    - Replaces the page that has not been used for the **longest time**.

4. FIFO rule:
    - Replaces the page that entered **cache earliest**.

Output:

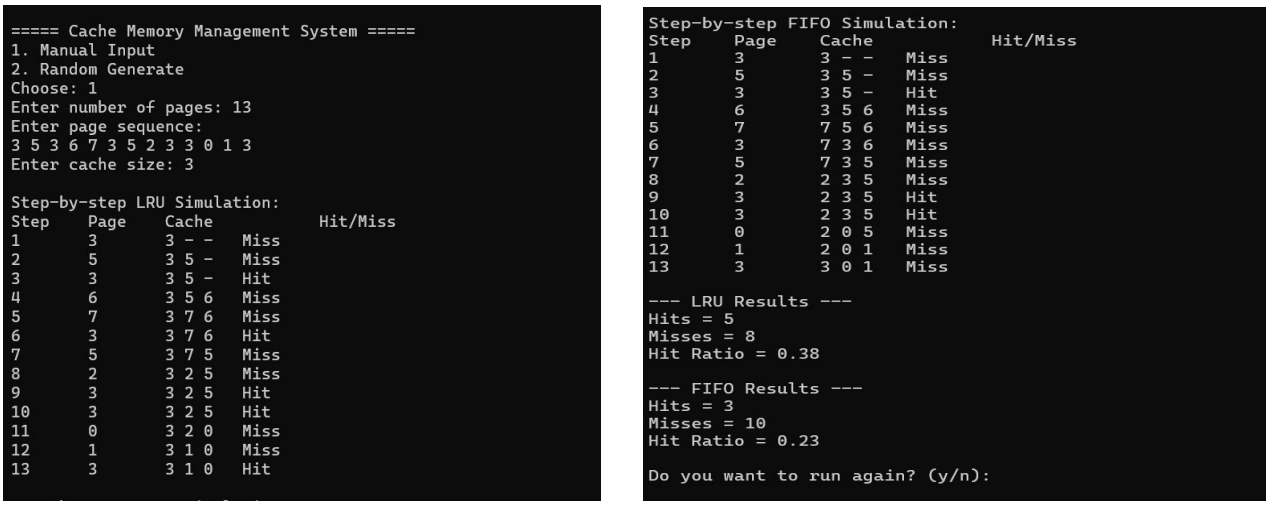
LRU -> Hits = 2, Misses = 10, Hit Ratio = 0.17

FIFO -> Hits = 3, Misses = 9, Hit Ratio = 0.25

Explanation:

- Both algorithms process the same reference string but choose different victim pages on a miss.
- In this test case, FIFO keeps pages that are reused soon more often than LRU, so FIFO gets one extra hit.
- Therefore, FIFO performs better for this specific input (‘0.25 > 0.17’).
- This confirms the program correctly distinguishes cache behaviour by policy and reports consistent hit/miss totals.

Test Case 2



```
===== Cache Memory Management System =====
1. Manual Input
2. Random Generate
Choose: 1
Enter number of pages: 13
Enter page sequence:
3 5 3 6 7 3 5 2 3 3 0 1 3
Enter cache size: 3

Step-by-step LRU Simulation:
Step    Page    Cache          Hit/Miss
1       3       3 - -   Miss
2       5       3 5 -   Miss
3       3       3 5 -   Hit
4       6       3 5 6   Miss
5       7       3 7 6   Miss
6       3       3 7 6   Hit
7       5       3 7 5   Miss
8       2       3 2 5   Miss
9       3       3 2 5   Hit
10      3       3 2 5   Hit
11      0       3 2 0   Miss
12      1       3 1 0   Miss
13      3       3 1 0   Hit

Step-by-step FIFO Simulation:
Step    Page    Cache          Hit/Miss
1       3       3 - -   Miss
2       5       3 5 -   Miss
3       3       3 5 -   Hit
4       6       3 5 6   Miss
5       7       7 5 6   Miss
6       3       7 3 6   Miss
7       5       7 3 5   Miss
8       2       2 3 5   Miss
9       3       2 3 5   Hit
10      3       2 3 5   Hit
11      0       2 0 5   Miss
12      1       2 0 1   Miss
13      3       3 0 1   Miss

--- LRU Results ---
Hits = 5
Misses = 8
Hit Ratio = 0.38

--- FIFO Results ---
Hits = 3
Misses = 10
Hit Ratio = 0.23

Do you want to run again? (y/n):
```

Fig 2. Output of Test Case 2 for LRU and FIFO

Output Explanation:

Input:

Mode: Manual

Number of pages: 13

Page sequence: 3 5 3 6 7 3 5 2 3 3 0 1 3

Cache size: 3

Process:

1. The program runs the same input through both replacement algorithms.
2. For each page reference, it checks whether the page already exists in cache.
    - If yes: count as Hit
    - If not: count as Miss and replace one page based on algorithm rule.
3. LRU rule :
    - Replaces the page that has not been used for the longest time.
4. FIFO rule:

- Replaces the page that entered cache earliest.

Output:

LRU -> Hits = 5, Misses = 8, Hit Ratio = 0.38

FIFO -> Hits = 3, Misses = 10, Hit Ratio = 0.23

Explanation:

- Both algorithms process the same reference string but choose different victim pages on a miss.
- In this test case, LRU keeps pages that are reused soon more often than FIFO, particularly noticing how it handles the frequent re-occurrence of page 3.
- Therefore, LRU performs better for this specific input (0.38 > 0.23).
- This confirms the program correctly distinguishes cache behaviour by policy and reports consistent hit/miss totals.

Test Case 3

Fig 3. Output of Test Case 3 for LRU and FIFO

Output Explanation:

Input:

Mode: Random Generate

Number of pages: 10

Page sequence: 0 7 1 7 1 6 4 2 8 8

Cache size: 3

Process:

1. The program runs the same input through both replacement algorithms.
2. For each page reference, it checks whether the page already exists in cache.
    - If yes: count as Hit
    - If not: count as Miss and replace one page based on algorithm rule.
3. LRU rule:
    - Replaces the page that has not been used for the longest time.
4. FIFO rule:
    - Replaces the page that entered cache earliest.

Output:

LRU -> Hits = 3, Misses = 7, Hit Ratio = 0.30 FIFO

-> Hits = 3, Misses = 7, Hit Ratio = 0.30

Explanation:

- Both algorithms process the same reference string but choose different victim pages on a miss.
- In this test case, LRU and FIFO result in the exact same number of hits because the frequency and order of page re-entry align for both policies.
- Therefore, both algorithms perform equally for this specific input (0.30 = 0.30).
- This confirms the program correctly distinguishes cache behavior by policy and reports consistent hit/miss totals, even when outcomes are identical.

7. Result Analysis

This section analyzes the results of LRU and FIFO **cache replacement policies** through various test cases. The performance of the two algorithms is **compared** based on hit, miss and hit ratio.
Also, their impact on **page access pattern**, **cache pollution** and **thrashing** is briefly discussed.

7.1 Test Case Comparison

A total of **three test cases** have been used in this project to compare the performance of **LRU** and **FIFO** cache replacement policies. In each test case, **two algorithms** have been run using the **same page sequence** and the **same cache size**, so that their performance can be analyzed properly.

In Test Case 1, the page sequence was **1 2 3 4 1 2 5 1 2 3 4 5** and the cache size was 3. In this case, the LRU algorithm provided 2 hits, 10 misses and a **0.17** hit ratio. On the other hand, the FIFO algorithm showed 3 hits, 9 misses and a **0.25** hit ratio. Therefore, FIFO gave a relatively better performance in this test case.

In Test Case 2, the page sequence was **3 5 3 6 7 3 5 2 3 3 0 1 3** and the cache size was 3. Here, the LRU algorithm achieved 5 hits, 8 misses and a **0.38** hit ratio. On the other hand, FIFO algorithm got 3 hits, 10 misses and **0.23** hit ratio. As a result, LRU clearly showed better results than FIFO in this test case.

In Test Case 3, the random page sequence was 0 7 1 7 1 6 4 2 8 8 and the cache size was 3. In this test case, both LRU and FIFO algorithms gave the same results. In both, 3 hits, 7 misses and **0.30 hit ratio** were found. That is, in this case, the two policies showed **equal performance**.

Therefore, comparing the three test cases, it can be seen that the same algorithm does not give the best results in all situations. Depending on the type of **page access pattern**, sometimes FIFO shows better performance, sometimes LRU shows better performance, and in some cases the results of the two algorithms may be the same.

7.2 Hit Ratio and Miss Ratio Analysis

This project uses hit, miss and hit ratio as the main indicators to evaluate cache performance. When the **requested page** is already in the cache, it is considered a **hit**. This allows data to be retrieved quickly and improves system performance. On the other hand, if the page is **not** in the cache, a miss occurs, and then cache replacement is required to **load a new page**, which **increases memory access time**.

Hit ratio is an important performance measure because it shows the **percentage** of **total page references** that the cache has successfully **served**. The **higher** the **hit** ratio, the more **efficient** the cache system is. Conversely, a **high miss ratio** means that the cache is forced to **replace pages** repeatedly, which **reduces** the overall efficiency of the system.

The results of this project show that hit, miss and hit ratio are directly dependent on the **page access pattern**. If the **same page** is used **repeatedly** in the page reference string, then the number of hits is more likely to **increase**. In such cases, performance is improved if the cache can retain recently or frequently used pages. Again, if the **page sequence** is such that **new pages** are accessed repeatedly, then the number of **misses increases** and the **hit ratio decreases**.

Therefore, the effectiveness of the cache replacement policy can be clearly understood through hit, miss and hit ratio analysis. Using these three measures, it is possible to determine which algorithm is relatively more efficient and under which circumstances performance is decreasing.

7.3 LRU vs FIFO Performance Discussion

Both LRU and FIFO are important replacement policies used for **fixed-size cache management**, but their working principle is not the same. **FIFO policy** removes the page that entered the cache first. This method is simpler and less complex, but it does not consider the recent use of the page. As a result, many times the necessary page may also be removed from the cache.On the other hand, **LRU policy** replaces the page that has not been used for the **longest time**. As a result, the recently used page is more likely to be in the cache. In real cases, where some pages are **reused** repeatedly, **LRU** usually gives **better performance**.

The test results of this project also show that LRU and FIFO **do not** always give the same performance. In some cases, FIFO gave better results, while in some cases, LRU achieved a higher hit ratio. The main reason for this is the difference in page reference patterns. If the **page access pattern** is such that the **old page** is needed again **quickly**, then **FIFO** can do better. But if the probability of **recently used pages** being reused is **high**, then **LRU** is relatively more **efficient**. So, while **LRU** is generally considered a **more intelligent policy** in terms of performance, the simplicity and low overhead of FIFO are also important advantages. So which policy is better depends entirely on the **input pattern** and **cache usage behavior**.

7.4 Cache Pollution and Thrashing Analysis

7.4.1 Cache Pollution

Cache pollution occurs when the cache contains pages that are **rarely used**, but there is **less space** for **useful pages**. This **reduces** the efficiency of the cache and reduces the hit ratio. The analysis of this project shows that the FIFO policy can often retain pages in the cache that are no longer used. This is because FIFO only follows the order in which the page is accessed, not whether the page is recently used or not. This can cause **useful pages** to be **removed** from the cache.

On the other hand, the **LRU policy** tries to retain recently used pages in the cache, which increases the likelihood of **less useful pages** being replaced. For this reason, LRU is generally more effective in **reducing cache pollution**. Especially when **some pages** are accessed repeatedly, LRU helps maintain better cache efficiency by retaining those pages.

7.4.2 Thrashing

Thrashing is a condition where **page replacements** occur **very frequently** in the cache and the number of **misses increases**. This problem is usually more common when the cache size is **small** or the page access pattern is not optimal. Thrashing forces the system to repeatedly **load new pages**, which **increases memory access time** and **reduces overall performance**.The test results of this

project show that the **higher the miss**, the higher the thrashing tendency. FIFO policy can cause more replacement in some cases, especially when old pages are needed again. LRU policy helps reduce thrashing in many cases by considering previous usage history. However, LRU is not the best for all input patterns.

Therefore, cache pollution and thrashing analysis show that replacement policy has a major impact on cache performance. By analyzing them properly, it is possible to easily determine which policy is more effective in which situation.

7.5 Overall Findings

- The performance of LRU and FIFO does not remain the **same** for **all test cases**.
- **Cache efficiency** depends greatly on the **page access pattern**.
- In some cases, **FIFO** gives better performance, while in other cases **LRU** performs better.
- LRU is generally more effective when **recently used pages** are accessed again.
- FIFO is simpler to implement, but it may **remove useful pages** without considering **recent usage**.
- LRU helps **reduce cache pollution** by keeping **recently used pages** in cache.
- **Thrashing** increases when **cache size is small** and page replacement happens **frequently**.
- Overall, **LRU** is often more **efficient**, but the **better policy** depends on the **input pattern** and **cache behaviour**.

7. Limitation

1. **Simplified Simulation Environment:** The project is implemented as a **basic simulation** and does not replicate real operating system memory management. It ignores important system-level aspects such as CPU scheduling, multitasking, and actual hardware interactions.
2. **Limited Algorithm Scope**: Only FIFO and LRU **page replacement algorithms** are considered. More advanced and optimal algorithms (Optimal, LFU, or Clock) are not included, which limits the depth of comparative analysis.

3. **Restricted Dataset and Input Size**: The evaluation is performed on relatively **small or randomly** generated page sequences. This may not accurately reflect real-world workloads where memory access patterns are more complex and large-scale.
4. **Limited Performance Metrics:** The analysis is primarily based on **hit and miss counts**. Other critical performance factors such as **execution time**, **latency**, and **computational overhead** are not considered.
5. **Lack of Graphical Visualization**: Although step-by-step outputs are displayed, the project does not include **graphical** or **interactive visualization tools**, which could make understanding cache behaviour more intuitive and insightful.
6. **Assumption of Ideal Conditions**: The implementation assumes ideal conditions without considering practical constraints such as limited processing power, memory overhead, or system interruptions.

8. Future Improvements

- **Additional Algorithms:** Add **Optimal Page Replacement** (OPT) or **Least Frequently Used (LFU)** algorithms to provide a more detailed comparison.
- **Dynamic Cache Sizing:** Have the program test different values of cache size to discover the best value to use with a particular page sequence.
- **Graphical User Interface (GUI):** Create a **graphical interface** that is easier to use and more intuitive to interpret cache blocks.
- **Error Handling:** Improve **error handling** by allowing input strings to be checked to avoid crashing on non-integer characters or invalid sequence lengths.
- **Automated Testing:** Have a collection of **known reference strings** to test the algorithm efficiency in known high-stress modes such as thrashing.
- **Performance Optimization:** The performance can be improved by **refactoring** the main simulation loops to be more memory efficient in the case of very large page reference strings.

9. Conclusion

The project manages to create a C program which mimics the management of the cache with the **Least Recently Used (LRU)** and **First In First Out (FIFO)** replacement policies. The step-bystep simulation is an important addition to the program features, as it enables users to see the current transitions of the cache in real-time and the reasoning behind the statuses of the "Hit" and "Miss" options. The **menu-based interactive** interface offers a simple method of entering page sequences either **manually** or **randomly**, select the **size** of the **cache**, and be summarized with real-time performance statistics. The program is useful in illustrating the **basic ideas** of **memory management**, such as **cache pollution** and **thrashing**. The project can be of value to learners of computer architecture by calculating and comparing the **ratios of hits** and thus providing a bridge between the theoretical aspects of memory concepts and their real-world applications. All in all, the project itself is an educational tool and a basis to build on, and its modular design allows making improvements effortlessly, like more complex replacement algorithms or advanced performance analytics.

10. Bibliography

To deepen my understanding of cache memory, replacement policies, and their performance analysis, I consulted several authoritative online resources. These references provided clear explanations, algorithms, and practical examples that supported the development of this project.

1. **GeeksforGeeks — LRU Cache Implementation** https://www.geeksforgeeks.org/system-design/lru-cache-implementation/
2. **GeeksforGeeks — FIFO Page Replacement Algorithm** https://www.geeksforgeeks.org/dsa/program-page-replacement-algorithms-set-2-fifo/
3. **GeeksforGeeks — Page Replacement Algorithms in Operating Systems** https://www.geeksforgeeks.org/operating-systems/page-replacement-algorithms-inoperating-systems/
4. **TutorialsPoint — Numerical on LRU and FIFO Algorithms** https://www.tutorialspoint.com/article/numerical-on-lru-fifo/